



南開大學
Nankai University

计算机学院

网络技术与应用实验报告

实验 5：简单路由器程序的设计

姓名：王旭

学号：2312166

专业：计算机科学与技术

2025 年 12 月 18 日

目录

1 实验要求	2
2 网络拓扑图	2
3 实验环境配置	2
3.1 环境选择	3
3.2 本机网卡 IP 设置	3
3.3 虚拟机 3(路由器 B)IP 网卡配置修改	5
3.4 虚拟机 3(路由器 B) 静态路由的添加	6
3.5 虚拟机的网络适配器配置	7
4 路由器程序设计	8
4.1 初始化模块	8
4.2 路由表管理模块	9
4.3 ARP 缓存管理模块	10
4.4 数据包处理模块	11
4.4.1 主处理函数 packet_handler	11
4.4.2 IPv4 数据包处理	11
4.4.3 ARP 数据包处理	11
4.5 用户交互模块	11
4.6 实验要求符合性检查	12
5 实验结果验证	12
5.1 初始化模块显示与选择网络设备	13
5.2 路由表管理测试	13
5.3 ARP 缓存表测试	14
5.4 主机 A 与主机 B 连通性测试	14
5.4.1 主机 A-> 主机 B	14
5.4.2 主机 B-> 主机 A	15

1 实验要求

1. 设计和实现一个路由器程序，要求完成的路由器程序能和现有的路由器产品（如思科路由器、华为路由器、微软的路由器等）进行协同工作。
2. 程序可以仅实现 IP 数据报的获取、选路、投递等路由器要求的基本功能。可以忽略分片处理、选项处理、动态路由表生成等功能。
3. 需要给出路由表的手工插入、删除方法。
4. 需要给出路由器的工作日志，显示数据报获取和转发过程。
5. 完成的程序须通过现场测试，并在班（或小组）中展示和报告自己的设计思路、开发和实现过程、测试方法和过程。

2 网络拓扑图

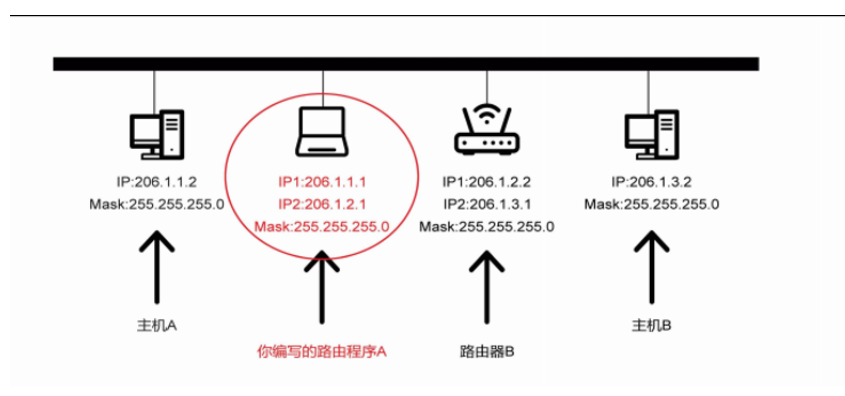


图 2.1: 网络拓扑图

3 实验环境配置

本次实验我认为有两大难点，第一个就是程序的设计，第二个就是实验环境的配置，并且这两个难点很难说到底谁更难，因为程序的设计运行与环境配置相互交织。在我进行本次实验的过程中，其中遇到了一个困扰我很久的问题（而这个问题后续发现其实是因为环境配置问题才得以成功解决，而不是程序问题）：

- 在对程序进行不断优化修改后，在实际的进行 ping 测试时，在其他两机之间互相 ping，都能够 ping 通并在程序中得到正确的捕获与转发结果，但是对于主机 A 与主机 B 而言，虽然能够正常从主机 A 向主机 B ping 通，但是 TTL 值不对，并且我发现程序运行结果中出现了源源不断的数据包与转发，这说明存在路由环路的问题

在经过不断地调试与分析之后，我才发现是因为环境配置问题（路由器 B 网卡设置 + VMware 网络适配器配置修改），才得以成功解决。

因此，在本次实验中，我们需要首先详细地来阐述本次实验的环境配置问题，这样才能正确地完成本次实验。

3.1 环境选择

本次实验我们有几个可选环境：

- 对于**测试环境**: 实验室机房电脑/**虚拟机环境**

由于实验室机房电脑的不稳定性，以及并没有对实验室机房电脑进行配置，以及我们无法像虚拟机那样，方便地进行随时的测试，因此我们必然选择的是 **虚拟机环境**。

如果我们选择虚拟机环境，那么我们需要把本机编译好的路由器可执行.exe 文件传入到作为路由器 A 的虚拟机 2 中，进行执行。但是由于所提供的虚拟机都是 **Windows Server2003 版** 的，由于虚拟机环境版本过于远古，与我们通过现有系统 API 等编译出来的.exe 可执行文件不兼容、不支持。因此通常情况下，我们在本机所编写好的路由器程序编译后的.exe 文件，通常无法在虚拟机上运行。

这就引出了一个问题：我们如何将我们在现在系统下编译出来的.exe 文件，能够在虚拟机 2(作为路由器 A) 上运行。为此我们有如下两个开发环境的选择，两种方式选一种来解决这个问题：

- 对于**开发环境**: Visual Studio/**VSCode**

两种开发环境下各自有方法来解决这个问题：

- 使用 Visual Studio 作为开发环境。依次点击项目-> 属性-> 链接器-> 系统-> 所需最低版本，在所需最低版本里面填写 5.01，并设置编译选项为 Release 和 x86。这样，不出意外的话 (或者运气好的话)，你的路由程序就可以在虚拟环境中成功运行。
- 使用 VSCode 作为开发环境。由于 VSCode 不像 Visual Studio 那样提供了这种降低版本的编译，因此我们如果使用 VSCode，那么所编译出来的程序应该还是无法在虚拟机中运行的。因此我们需要另辟蹊径：将我们个人主机的虚拟网卡 VMnet8 作为“路由器 A”，代替虚拟机 2 的位置，这样我们编译出的程序就必然可以在“路由器 A”上运行了。但这样需要额外修改 VMware 的配置，我们后续详细解释。

由于我们之前的开发环境 (前几个实验中的编程实验 lab2 和 lab3) 都是在 VSCode 上进行开发的 (并且我也没有安装 Visual Studio)，因此我们选择 **VSCode** 进行开发。

那么我们在之前 NPcap 环境的基础上，进行进一步配置。在之前的 NPcap 环境中，我们采用的是 Windows 环境下: **VSCode + MinGW/GCC + NPcap** 的开发方式。

- 注: 这个 Npcap 的环境配置你可以参考我的[实验二 github 仓库](#)

3.2 本机网卡 IP 设置

我们首先来进行本机 VMnet8 网卡的设置，来调整为网络拓扑图中“路由器 A(虚拟机 2) 的 IP 配置”。我们在这个单网卡上进行双 IP 配置。

我们在本机点击 **控制面板 > 网络和 Internet > 网络和共享中心**，然后在这个界面点击“**更改适配器设置**”，进入本机的各网卡 (网络连接) 页面。

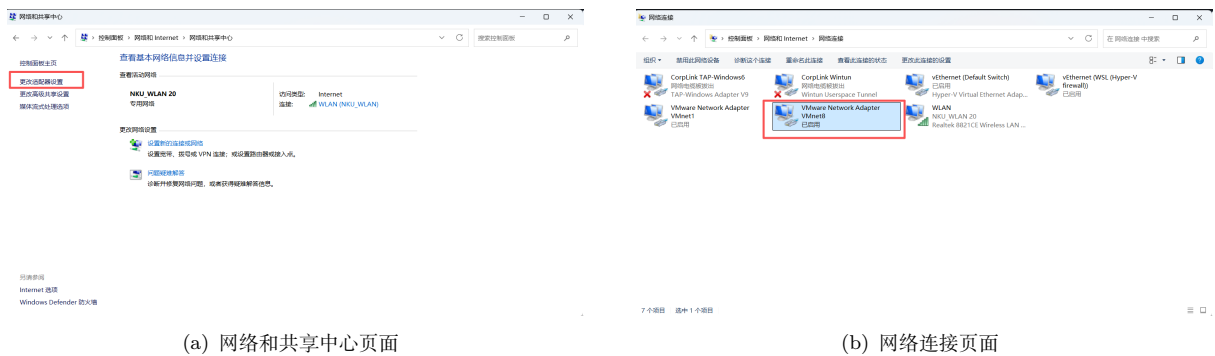
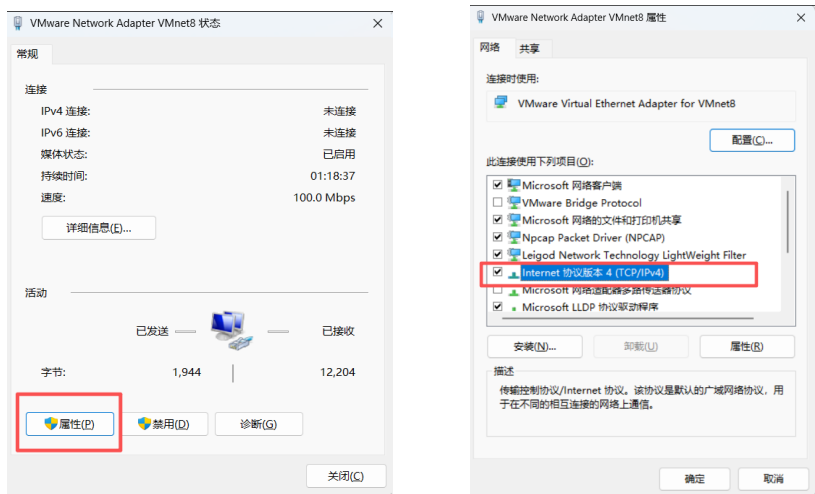


图 3.2: 更改本机 VMnet8 网卡配置

然后双击我们的”VMware Network Adapter VMnet8” 网卡进入具体的 IP 配置。



然后修改 VMware Network Adapter VMnet8 的 IPv4 的 IP 配置:



这样我们就完成了本机 VMnet8 的网卡配置了，除此之外对于本机不需要修改其他配置了。

- 注意，我们在本机不需要通过注册表修改 `IPEnableRouter` 值为 1 来打开” 路由器转发功能”。这是因为我们要实现的程序本身就要实现” 路由器转发功能” 等路由器的基本功能，来作为一个路由器，如果我们在本机还打开了 `IPEnableRouter`，那么在我们的实验网络中就会存在多个路由器，导致数据包转发混乱，甚至出现路由环路的情况，因此我们不需要打开 `IPEnableRouter`。

3.3 虚拟机 3(路由器 B)IP 网卡配置修改

在我们所给的虚拟机 3(路由器 B) 中，是同样单网卡双 IP 的网卡配置，在虚拟机 3 的本地连接 1 中配置了两个 IP:206.1.2.2 和 206.1.3.1。但是为了实现不同网络之间的具体层次，即 206.1.1.0 与 206.1.2.0 和 206.1.3.0 之间的具体层次，我们需要将虚拟机 3(路由器 B) 的两个 IP 分开到两个本地连接(网卡)上，这样我们后面在” 虚拟机网络适配器的配置” 中，才能实现正确的网络配置分配。这个我们后面在” 虚拟机网络适配器的配置” 部分会详细讲。

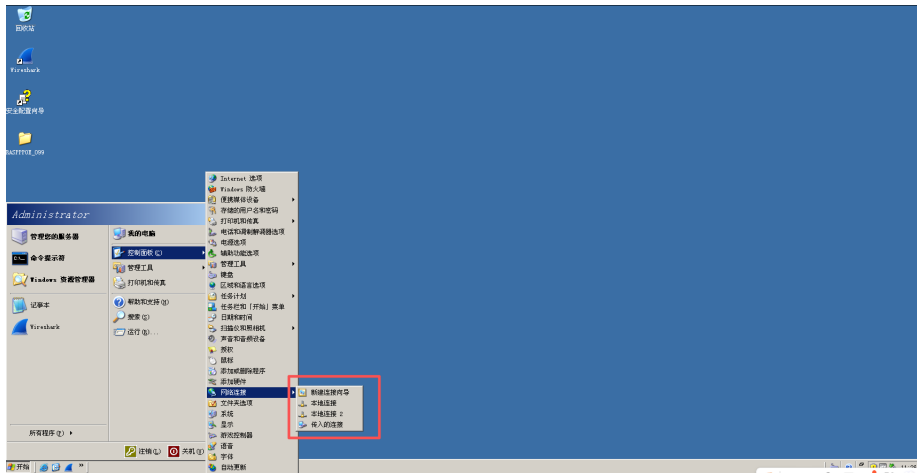


图 3.5: 修改虚拟机 3(路由器 B)IP 网卡配置

我们按上图中的路径点开” 本地连接”，然后将其中的双 IP 配置改为单 IP，即删除其中的 206.1.3.1，保留 206.1.2.2。如下图所示：

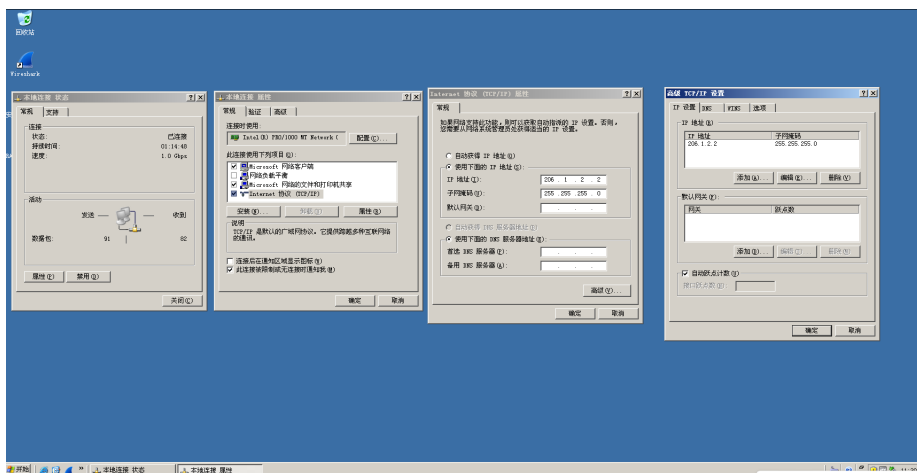


图 3.6: 本地连接 1 对应 IP 206.1.2.2

然后再按图 2.4 的路径打开 (启用)“本地连接 2”，然后进行配置 IP，配置路由器 B 的另一个 IP 206.1.3.1，如下图所示：

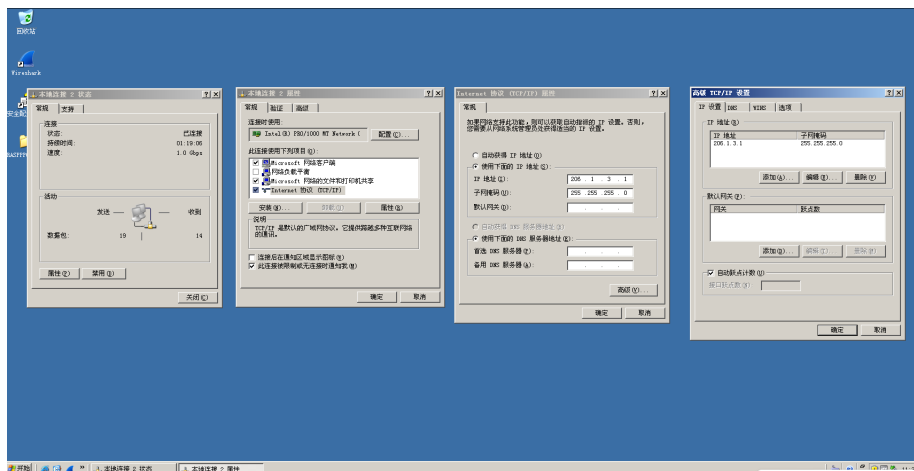


图 3.7: 本地连接 2 对应 IP 206.1.3.1

这样我们就完成了虚拟机 3(路由器 B) 的网卡配置修改，将原来的”单网卡双 IP”修改为了”双网卡双 IP”，这样才得以让 206.1.2.2 与 206.1.3.1 分开，为了对应后面的 206.1.2.0 与 206.1.3.0 两个网络分开。

3.4 虚拟机 3(路由器 B) 静态路由的添加

比如我们从主机 B(206.1.3.2) 向主机 A(206.1.1.2) 通信，当我们数据包来到路由器 B 时，他不知道如果进入 206.1.1.0 这个网络，下一跳的路由器是哪个，因为我们的路由器 B 上只有两个直连路由，对应 206.1.2.0 和 206.1.3.0，因此我们需要在路由器 B 上添加静态路由，告诉他，进入 206.1.1.0 的流量，要交给路由器 A(206.1.2.1)。因此我们在虚拟机 3(路由器 B) 上添加这条静态路由：

```
1 route ADD 206.1.1.0 MASK 255.255.255.0 206.1.2.1
```

这样路由器 B 上的 route print 结果才能出现通往 206.1.1.0 的路。

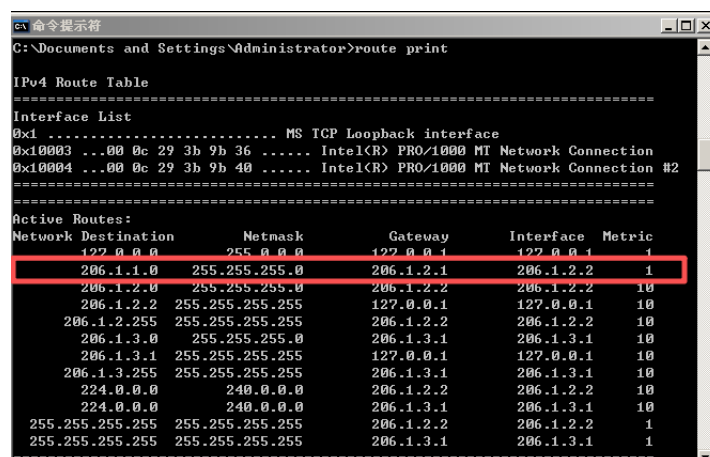


图 3.8: route print 结果

3.5 虚拟机的网络适配器配置

现在完成了主机 VMnet8 网卡以及三台虚拟机的 IP 配置，我们现在需要把三台虚拟机与我们的主机的 VMnet8 网卡建立联系。

我们按照下面的表格，在 VMware WorkStation 上进行三台虚拟机的网络适配器配置。

设备名称	IP 地址	子网掩码	网络适配器设置
主机 A	206.1.1.2	255.255.255.0	VMnet8 (NAT 模式)
路由器 A	206.1.1.1	255.255.255.0	本主机的 VMnet8 网卡 (配置双 IP)
	206.1.2.1	255.255.255.0	
路由器 B	206.1.2.2	255.255.255.0	本地连接 1: VMnet8 (NAT 模式)
	206.1.3.1	255.255.255.0	本地连接 2: VMnet1 (仅主机模式)
主机 B	206.1.3.2	255.255.255.0	VMnet1 (仅主机模式)

表 1: 网络拓扑设备配置表

按照这样的配置方式，我们才能正确配置网络对应关系，如下：

- 将主机 A 配置到 VMnet8，这样主机 A(206.1.1.2) 就能够与我们的主机 (路由器 A) 的 206.1.1.1 在同一网络下。
- 将路由器 B 的本地连接 1(206.1.2.2) 配置到 VMnet8，这样路由器 B 的 206.1.2.2 就能够与我们的主机 (路由器 A) 的 206.1.2.1 在同一网络下。
- 将路由器 B 的本地连接 2(206.1.3.1) 配置到 VMnet1，这样路由器 B 的 206.1.3.1 就能够和”主机 A 与路由器 A 所在的网络”即 206.1.1.0 与 206.1.2.0 分开在不同网络下。而在一个单独的网络中。
- 将主机 B 配置到 VMnet1，这样主机 B 的 206.1.3.2 就能够与路由器 B 的 206.1.3.1 在同一网络下，同时与另外两个网络 206.1.1.0 与 206.1.2.0 分开。

只有将虚拟机 3(路由器 B) 的双 IP 分开在两个网卡上，我们才能够对这两个网卡分别进行网络适配器的配置，从而将这两个网络分开，如果我们还沿用原来的”单网卡双 IP”，那么我们无法将”路由器 B 的 206.1.3.1 与主机 B(206.1.3.2)”这个网络单独成一个网络，从而导致三台虚拟机以及我们的 VMnet8 网卡都同存于同一个网络下，导致我们的路由器程序甚至会捕捉到”206.1.3.0”这个网络中的数据包，这也就是我们之前提到的路由混乱的原因。

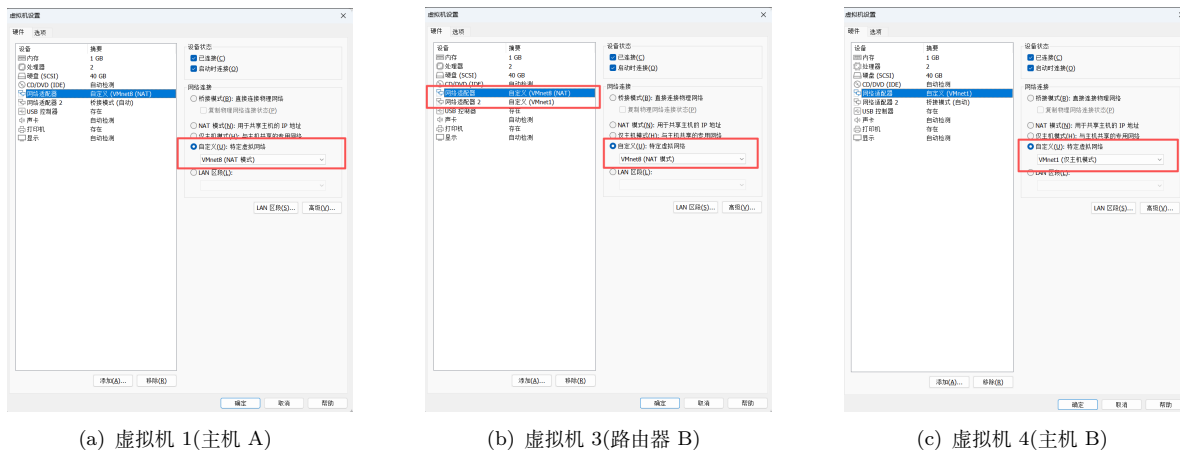


图 3.9: 更改三台虚拟机的网络适配器配置

因此这样我们本次实验的环境配置工作就全部完成了，下面就进行路由器程序的开发即可。只有首先配置好正确的环境，我们的工作才能事半功倍，否则就会像我之前遇到的路由混乱问题那样，久久无法解决。

4 路由器程序设计

我们本实验所实现的程序实现了一个用户态的软件路由器，基于 Npcap 库在 Windows 平台上运行。主要功能包括：

- **数据包捕获与解析：**实时捕获网络数据包，解析以太网、IPv4、ARP、ICMP 等协议
- **路由转发决策：**根据路由表进行 IP 数据包转发决策
- **ARP 协议处理：**实现 ARP 请求响应和 ARP 缓存表的管理
- **ICMP 协议支持：**响应 ICMP Echo 请求
- **路由表管理：**支持动态添加、删除和显示路由表项
- **网络状态监控：**实时显示数据包处理日志和网络状态

下面我们以**模块形式**讲解一下我们代码中的关键设计部分。我们将程序分为以下几个模块：

- 初始化模块
- 路由表管理模块
- ARP 缓存管理模块
- 数据包处理模块
- 用户交互模块

4.1 初始化模块

功能：选择网卡，获取本机的双 IP 地址和 MAC 地址，并自动添加默认路由。

代码实现：使用 `pcap_findalldevs` 获取所有网卡设备，让用户选择。通过 `pcap_open_live` 打开选中的网卡。从网卡信息中获取 IP 地址 (`local_ips_all`) 和 MAC 地址 (`g_local_mac`)。为每个本机 IP 地址添加一条直连路由（默认路由）。

获取本机 IP 地址并添加直连路由

```
1 // 获取本机 IP 地址
2 for (pcap_addr* a = device->addresses; a != nullptr; a = a->next) {
3     if (a->addr && a->addr->sa_family == AF_INET) {
4         sockaddr_in sin = (sockaddr_in)a->addr;
5         array<unsigned char, 4> ip_bytes;
6         memcpy(ip_bytes.data(), &sin->sin_addr, 4);
7         local_ips_all.push_back(ip_bytes);
8         .....
9     }
```

```

10 // 添加直连路由
11 for (const auto& ip : local_ips_all) {
12     unsigned char net[4], mask[4] = { 255, 255, 255, 0 };
13     memcpy(net, ip.data(), 4);
14     net[3] = 0;
15     add_route(net, mask, ip.data(), true);
16 }
17 }

```

4.2 路由表管理模块

功能：实现路由表的显示、添加和删除。

代码实现：路由表使用 `vector<RouteEntry>` 存储。`add_route` 函数添加路由，包括直连路由（默认）和手动添加的路由。`del_route` 函数删除路由，但不能删除默认路由（通过 `is_default` 标志判断）。`show_routing_table` 函数显示当前路由表。`manual_add/delete_route` 函数手动添加/删除路由。

添加路由

```

1 // 添加路由条目
2 void add_route(const unsigned char network[4], const unsigned char netmask[4], const
   unsigned char gateway[4], bool is_default = false) {
3     RouteEntry entry;
4     memcpy(entry.network.data(), network, 4);
5     memcpy(entry.netmask.data(), netmask, 4);
6     memcpy(entry.gateway.data(), gateway, 4);
7     entry.is_default = is_default;
8     routing_table.push_back(entry);
9     cout << "[路由] 添加路由: " << entry.to_string() << endl;
10 }

```

删除路由

```

1 // 删除路由条目
2 void del_route(const unsigned char network[4], const unsigned char netmask[4]) {
3     for (auto it = routing_table.begin(); it != routing_table.end(); ) {
4         if (it->is_default) {
5             ++it;
6             continue;
7         }
8         if (memcmp(it->network.data(), network, 4) == 0 && memcmp(it->netmask.data(),
   netmask, 4) == 0) {
9             cout << "[路由] 删除路由: " << it->to_string() << endl;
10            it = routing_table.erase(it);
11            return;
12        }
13        else {
14            ++it;

```

```

15     }
16 }
17 cout << "[路由] 未找到匹配的路由项或路由项为默认路由,删除失败。" << endl;
18 }

```

4.3 ARP 缓存管理模块

功能：管理 ARP 缓存，包括添加、更新、查询和显示。

代码实现：使用 `map<array<unsigned char, 4>, ArpCacheEntry>` 存储 ARP 缓存。在 ARP 缓存表内容方面使用 `update_or_add_cache_entry` 函数更新或添加 ARP 缓存项。`find_mac_in_cache` 函数查询 ARP 缓存。`print_arp_cache` 函数显示 ARP 缓存表。

查询 ARP 缓存

```

1 // 在 ARP 缓存中查找 MAC 地址
2 bool find_mac_in_cache(const unsigned char ip[4], unsigned char mac_out[6]) {
3     lock_guard<mutex> lock(arp_cache_mutex); // 线程安全锁
4     array<unsigned char, 4> key_ip;
5     memcpy(key_ip.data(), ip, 4);
6     auto it = arp_cache.find(key_ip);
7     if (it != arp_cache.end() && it->second.valid) {
8         memcpy(mac_out, it->second.mac.data(), 6);
9         return true;
10    }
11    return false;
12 }

```

更新或添加 ARP 缓存

```

1 // 更新或添加 ARP 缓存条目
2 void update_or_add_cache_entry(const unsigned char ip[4], const unsigned char mac[6])
3 {
4     lock_guard<mutex> lock(arp_cache_mutex); // 线程安全锁
5     array<unsigned char, 4> key_ip;
6     array<unsigned char, 6> value_mac;
7     memcpy(key_ip.data(), ip, 4);
8     memcpy(value_mac.data(), mac, 6);
9     ArpCacheEntry entry;
10    entry.ip = key_ip;
11    entry.mac = value_mac;
12    entry.last_update = chrono::steady_clock::now();
13    entry.valid = true;
14    arp_cache[key_ip] = entry;
15    cout << "[ARP缓存] 更新: " << entry.to_string() << endl;
16 }

```

4.4 数据包处理模块

这是最核心的模块，包括 IP 数据包和 ARP 数据包的处理。

4.4.1 主处理函数 packet_handler

首先解析以太网头部，根据以太网类型区分 IPv4 和 ARP。忽略源 MAC 为本机 MAC 的数据包（避免处理自己发送的数据包）。

4.4.2 IPv4 数据包处理

解析 IP 头部，获取源 IP、目的 IP、TTL 和协议类型。如果目的 IP 是本机，则检查协议类型，如果是 ICMP 则调用 `handle_icmp_echo_request` 处理。如果目的 IP 不是本机，则进行转发：查找路由表，找到下一跳。检查 TTL，如果 $TTL \leq 1$ 则丢弃。TTL 减 1，重新计算 IP 校验和。查询下一跳的 MAC 地址（先查 ARP 缓存，如果没有则发送 ARP 请求）。修改以太网头部，设置目的 MAC 为下一跳 MAC，源 MAC 为本机 MAC，然后发送。

4.4.3 ARP 数据包处理

如果是 ARP 请求 (Request)，并且目标 IP 是本机，则构造 ARP 回复 (Reply) 并发送。如果是 ARP 回复，则更新 ARP 缓存。

IPv4 数据包转发部分

```
1 RouteEntry* route = find_route(dst_ip);
2 if (route) {
3     // ... TTL处理
4     // ARP查询
5     if (find_mac_in_cache(arp_target_ip, next_hop_mac)) {
6         // 使用缓存
7     } else {
8         // 发送ARP请求
9         send_arp_and_wait_reply(g_handle, g_local_mac, arp_local_ip_final,
10                                arp_target_ip, next_hop_mac);
11     }
12     // 发送数据包
13 }
```

ARP 请求处理

```
1 if (ntohs(arp->oper) == 1) { // Request
2     if (is_local_ip(arp->tpa)) {
3         // 构造ARP Reply并发送
4     }
5 }
```

4.5 用户交互模块

功能：提供命令行菜单，让用户选择不同的功能。

代码实现：主循环中显示菜单，根据用户输入执行相应操作。功能包括启动路由器、显示路由表、显示 ARP 缓存、手动添加路由、手动删除路由。

用户交互菜单

```
1 while (true) {
2     cout << "\n 选择功能 \n";
3     cout << "1. 启动路由器 (抓包并转发) \n";
4     cout << "2. 显示路由表\n";
5     cout << "3. 显示 ARP 缓存\n";
6     cout << "4. 手动添加路由\n";
7     cout << "5. 手动删除路由\n";
8     cout << "0. 退出程序\n";
9     // ... 处理用户输入
10 }
```

4.6 实验要求符合性检查

- 获取并显示本机单网卡的双 IP 地址和 MAC 地址：在初始化模块中实现，选择网卡后显示。
- 路由表管理：通过路由表管理模块实现显示、添加和删除。
- 工作日志：在数据包处理过程中，使用 `cout` 输出日志信息，包括源 IP、目的 IP、下一跳等。
- 连通性测试：通过处理 ICMP 请求和转发 IP 数据包，实现主机 A 和 B 的 ping 通及 TTL 处理正确。

5 实验结果验证

要能够正确处理主机 A(206.1.1.2) 与主机 B(206.1.3.2) 之间的成功通信，我们首先要保证通路顺畅完整，要记得：

- 在虚拟机 3(路由器 B) 上添加静态路由 `route ADD 206.1.1.0 MASK 255.255.255.0 206.1.2.1`，这样保证 路由器 B-> 路由器 A 通路的连接。即 路由器 B->206.1.1.0 网络 的通路。
- 在我们的路由器程序上添加路由 `206.1.3.0/255.255.255.0/206.1.2.2`，保证 路由器 A-> 路由器 B 通路的连接。即 路由器 A->206.1.3.0 网络 的通路。

5.1 初始化模块显示与选择网络设备

```
PS D:\VS Code\NETMORO> ./bin/version13.exe
=== Npcap 简化路由程序 ===

可用的网络设备:
1. \Device\NPF_{910F7D40-50B3-4852-8794-2A74FC8A1661} - WAN Miniport (Network Monitor)
2. \Device\NPF_{6526F4B3-23F6-4839-BA2A-450E6A845412} - WAN Miniport (IPv6)
3. \Device\NPF_{22086905-76CA-446A-8F57-DF5B9B162FD} - WAN Miniport (IP)
4. \Device\NPF_{764C5D46-87A8-470C-9F43-D272C25962C8} - Hyper-V Virtual Ethernet Adapter #2
5. \Device\NPF_{864E87A8-468E-4506-9836-A9409F1ED243} - Hyper-V Virtual Ethernet Adapter
6. \Device\NPF_{0A887C94-9477-4875-8593-978E42117445} - Realtek 802.11E Wireless LAN 802.11ac PCI-E NIC
7. \Device\NPF_{B114056E-1B81-47FE-8230-EDC2A02752C4} - VMware Virtual Ethernet Adapter for VMnet8
8. \Device\NPF_{80050F67-9681-404D-9710-DAF67A5A025} - VMware Virtual Ethernet Adapter for VMnet1
9. \Device\NPF_{D907DE50-7F32-4366-B926-E7BC13B03530} - Microsoft Wi-Fi Direct Virtual Adapter #2
10. \Device\NPF_{56D1FF55-2E15-490C-8000-0895383CA1FC} - Microsoft Wi-Fi Direct Virtual Adapter
11. \Device\NPF_{Loopback} - Adapter for loopback traffic capture
12. \Device\NPF_{143C9220-8B62-4871-A20C-B23A88E19746} - TAP-Windows Adapter V9
13. \Device\NPF_{C9BC7C9-69B2-462F-A208-EA367CAEDF68} - Minton Userspace Tunnel

请选择要使用的设备编号 (1-13):
```

(a) 获取网络设备列表

```
请选择要使用的设备编号 (1-13): 7
选择设备: VMware Virtual Ethernet Adapter for VMnet8
本机 IPv4 地址列表:
206.1.2.1
206.1.1.1
[路由] 添加路由: 206.1.2.0/255.255.255.0 -> 206.1.2.1
[路由] 添加路由: 206.1.1.0/255.255.255.0 -> 206.1.1.1
本机 MAC: 00:50:56:c0:00:08

===== 选择功能 =====
1. 启动路由器 (抓包并转发)
2. 显示路由表
3. 显示 ARP 缓存
4. 手动添加路由
5. 手动删除路由
0. 退出程序
输入编号:
```

(b) 选择 VMnet8 网卡

图 5.10: 初始化模块运行界面

可以看到我们首先成功获取了我们本机的网络设备列表，然后在我们选择了我们本机的第 7 个设备 VMware Virtual Ethernet Adapter for VMnet8 之后，程序成功输出了本机 VMnet8 网卡的两个 IP 地址:206.1.1.1 和 206.1.2.1，并成功自动地添加了这两条直连默认路由。最后输出了本机的 MAC 地址。

5.2 路由表管理测试

此时我们选择：**2. 显示路由表功能**，我们可以看到路由表中目前只有两个直连路由，如下：

```
===== 选择功能 =====
1. 启动路由器 (抓包并转发)
2. 显示路由表
3. 显示 ARP 缓存
4. 手动添加路由
5. 手动删除路由
0. 退出程序
输入编号: 2

===== 路由表 =====
206.1.2.0/255.255.255.0 -> 206.1.2.1 (直连路由(默认))
206.1.1.0/255.255.255.0 -> 206.1.1.1 (直连路由(默认))
=====
```

图 5.11: 初始路由表结果

我们尝试删除这两个直连路由，发现无法删除，如下：

```
===== 选择功能 =====
1. 启动路由器 (抓包并转发)
2. 显示路由表
3. 显示 ARP 缓存
4. 手动添加路由
5. 手动删除路由
0. 退出程序
输入编号: 5
请输入要删除的路由信息 (格式: 网络/掩码, 例如 206.1.3.0/255.255.255.0):
206.1.2.0/255.255.255.0
[路由] 未找到匹配的路由项或路由项为默认路由,删除失败。

===== 选择功能 =====
1. 启动路由器 (抓包并转发)
2. 显示路由表
3. 显示 ARP 缓存
4. 手动添加路由
5. 手动删除路由
0. 退出程序
输入编号: 2

===== 路由表 =====
206.1.2.0/255.255.255.0 -> 206.1.2.1 (直连路由(默认))
206.1.1.0/255.255.255.0 -> 206.1.1.1 (直连路由(默认))
=====
```

图 5.12: 禁止删除默认路由

下面我们添加路由 206.1.3.0/255.255.255.0/206.1.2.2，成功添加，如下：

```

===== 选择功能 =====
1. 启动路由器（抓包并转发）
2. 显示路由表
3. 显示 ARP 缓存
4. 手动添加路由
5. 手动删除路由
0. 退出程序
输入编号: 4
请输入要添加的路由信息（格式：网络/掩码/网关，例如 206.1.3.0/255.255.255.0/206.1.2.2）：
206.1.3.0/255.255.255.0/206.1.2.2
[路由] 添加路由: 206.1.3.0/255.255.255.0 -> 206.1.2.2

===== 选择功能 =====
1. 启动路由器（抓包并转发）
2. 显示路由表
3. 显示 ARP 缓存
4. 手动添加路由
5. 手动删除路由
0. 退出程序
输入编号: 2

===== 路由表 =====
206.1.2.0/255.255.255.0 -> 206.1.2.1 (直连路由(默认))
206.1.1.0/255.255.255.0 -> 206.1.1.1 (直连路由(默认))
206.1.3.0/255.255.255.0 -> 206.1.2.2
=====

```

图 5.13: 添加下一跳路由 B，通往 206.1.3.0

5.3 ARP 缓存表测试

此时我们选择：**3. 显示 ARP 缓存**功能，我们可以看到缓存表中目前什么也没有，为空，因为我们还没有进行任何的 ping 测试。如下：

```

===== 选择功能 =====
1. 启动路由器（抓包并转发）
2. 显示路由表
3. 显示 ARP 缓存
4. 手动添加路由
5. 手动删除路由
0. 退出程序
输入编号: 3

===== ARP 缓存表 =====
(空)
=====

```

图 5.14: 初始 ARP 缓存表结果

后续我们进行完主机 A(206.1.1.2) 向主机 B(206.1.3.2) 的 ping 测试之后，ARP 缓存表在过程中得到更新，结果为：

```

===== 选择功能 =====
1. 启动路由器（抓包并转发）
2. 显示路由表
3. 显示 ARP 缓存
4. 手动添加路由
5. 手动删除路由
0. 退出程序
输入编号: 3

===== ARP 缓存表 =====
206.1.1.2 -> 00:0c:29:3b:19:dd
206.1.2.2 -> 00:0c:29:3b:9b:36
=====

```

图 5.15: ARP 缓存表在 ping 过程中得到更新

5.4 主机 A 与主机 B 连通性测试

在启动程序的功能 **1:1. 启动路由器（抓包并转发）** 之后，我们开始 ping 测试。

5.4.1 主机 A-> 主机 B

首先从主机 A 向主机 B 发送 ping 命令，结果如下：

```
命令提示符
Microsoft Windows [版本 5.2.3790]
(C) 版权所有 1985-2003 Microsoft Corp.

C:\Documents and Settings\Administrator>ping 206.1.3.2

Pinging 206.1.3.2 with 32 bytes of data:

Reply from 206.1.3.2: bytes=32 time=3719ms TTL=126
Reply from 206.1.3.2: bytes=32 time=2035ms TTL=126
Reply from 206.1.3.2: bytes=32 time=2033ms TTL=126
Reply from 206.1.3.2: bytes=32 time=2028ms TTL=126

Ping statistics for 206.1.3.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2028ms, Maximum = 3719ms, Average = 2453ms

C:\Documents and Settings\Administrator>
```

图 5.16: 主机 A ping 主机 B 结果

可以看到我们成功 ping 通，并且 TTL 值为 126，验证了我们程序的正确性。
路由器程序的抓包转发运行结果：

```
路由程序启动，按 Ctrl+C 停止并返回菜单。
1) [ARP] 源MAC: 00:0c:29:3b:19:dd, 目的MAC: ff:ff:ff:ff:ff:ff
[ARP] 收到 ARP Request: 206.1.1.2 请求 206.1.1.1
[ARP] 请求的目标是本地，正在构造 ARP Reply...
[ARP] ARP Reply 已发送给 206.1.1.2
2) [IPv4] 源IP: 206.1.1.2 -> 目的IP: 206.1.3.2 | TTL=128 | 协议=1 (下一跳: 206.1.2.2)
[转发] TTL -> 127
[转发] 将查询网关 (206.1.2.2) 的 MAC 地址。
[ARP缓存缺失] 正在查询 (206.1.2.2) 的 MAC 地址...
[ARP缓存] 更新: 206.1.2.2 -> 00:0c:29:3b:19:36
[ARP命中] 目标 MAC: 00:0c:29:3b:19:36
[转发] 数据包已发送
3) [IPv4] 源IP: 206.1.3.2 -> 目的IP: 206.1.1.2 | TTL=127 | 协议=1 (下一跳: 206.1.1.1)
[转发] TTL -> 126
[转发] 将直接查询目标主机 (206.1.1.2) 的 MAC 地址。
[ARP缓存缺失] 正在查询 (206.1.1.2) 的 MAC 地址...
[ARP缓存] 更新: 206.1.1.2 -> 00:0c:29:3b:19:dd
[ARP命中] 目标 MAC: 00:0c:29:3b:19:dd
[转发] 数据包已发送
4) [IPv4] 源IP: 206.1.1.2 -> 目的IP: 206.1.3.2 | TTL=128 | 协议=1 (下一跳: 206.1.2.2)
[转发] TTL -> 127
[转发] 将查询网关 (206.1.2.2) 的 MAC 地址。
[ARP缓存命中] 目标 MAC: 00:0c:29:3b:19:36
[转发] 数据包已发送
```

```
5) [IPv4] 源IP: 206.1.3.2 -> 目的IP: 206.1.1.2 | TTL=127 | 协议=1 (下一跳: 206.1.1.1)
[转发] TTL -> 126
[转发] 将直接查询目标主机 (206.1.1.2) 的 MAC 地址。
[ARP缓存命中] 目标 MAC: 00:0c:29:3b:19:dd
[转发] 数据包已发送
6) [IPv4] 源IP: 206.1.1.2 -> 目的IP: 206.1.3.2 | TTL=128 | 协议=1 (下一跳: 206.1.2.2)
[转发] TTL -> 127
[转发] 将查询网关 (206.1.2.2) 的 MAC 地址。
[ARP缓存命中] 目标 MAC: 00:0c:29:3b:19:36
[转发] 数据包已发送
7) [IPv4] 源IP: 206.1.3.2 -> 目的IP: 206.1.1.2 | TTL=127 | 协议=1 (下一跳: 206.1.1.1)
[转发] TTL -> 126
[转发] 将直接查询目标主机 (206.1.1.2) 的 MAC 地址。
[ARP缓存命中] 目标 MAC: 00:0c:29:3b:19:dd
[转发] 数据包已发送
8) [IPv4] 源IP: 206.1.1.2 -> 目的IP: 206.1.3.2 | TTL=128 | 协议=1 (下一跳: 206.1.2.2)
[转发] TTL -> 127
[转发] 将查询网关 (206.1.2.2) 的 MAC 地址。
[ARP缓存命中] 目标 MAC: 00:0c:29:3b:19:36
[转发] 数据包已发送
9) [IPv4] 源IP: 206.1.3.2 -> 目的IP: 206.1.1.2 | TTL=127 | 协议=1 (下一跳: 206.1.1.1)
[转发] TTL -> 126
[转发] 将直接查询目标主机 (206.1.1.2) 的 MAC 地址。
[ARP缓存命中] 目标 MAC: 00:0c:29:3b:19:dd
[转发] 数据包已发送
```

图 5.17: 路由器程序抓包及转发日志

可以看到我们的路由器程序的工作日志，正确显示了数据报获取和转发过程，展示了数据报的源 IP、目的 IP、下一跳信息。同时也正确处理了 ARP 缓存表的更新过程。

5.4.2 主机 B-> 主机 A

首先从主机 B 向主机 A 发送 ping 命令，结果如下：


```

C:\命令提示符
Reply from 206.1.1.2: bytes=32 time=3872ms TTL=126
Reply from 206.1.1.2: bytes=32 time=2828ms TTL=126
Reply from 206.1.1.2: bytes=32 time=2817ms TTL=126
Reply from 206.1.1.2: bytes=32 time=2824ms TTL=126

Ping statistics for 206.1.1.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2817ms, Maximum = 3872ms, Average = 2485ms

C:\Documents and Settings\Administrator>ping 206.1.1.2

Pinging 206.1.1.2 with 32 bytes of data:

Reply from 206.1.1.2: bytes=32 time=1740ms TTL=126
Reply from 206.1.1.2: bytes=32 time=2827ms TTL=126
Reply from 206.1.1.2: bytes=32 time=2840ms TTL=126
Reply from 206.1.1.2: bytes=32 time=2826ms TTL=126

Ping statistics for 206.1.1.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1740ms, Maximum = 2840ms, Average = 1958ms

C:\Documents and Settings\Administrator>
    
```

图 5.18: 主机 B ping 主机 A 结果

可以看到我们成功 ping 通，并且 TTL 值为 126，验证了我们程序的正确性。
路由器程序的抓包转发运行结果：

<pre> [10][IPv4] 源IP: 206.1.1.2 -> 目的IP: 206.1.3.2 TTL=128 协议=1 (下一跳: 206.1.2.2) [转发] TTL -> 127 [转发] 将查询网关 (206.1.2.2) 的 MAC 地址。 [ARP缓存命中] 目标 MAC: 00:0c:29:3b:9b:36 [转发] 数据包已发送 [11][IPv4] 源IP: 206.1.3.2 -> 目的IP: 206.1.1.2 TTL=127 协议=1 (下一跳: 206.1.1.1) [转发] TTL -> 126 [转发] 将直接查询目标主机 (206.1.1.2) 的 MAC 地址。 [ARP缓存命中] 目标 MAC: 00:0c:29:3b:19:dd [转发] 数据包已发送 [12][IPv4] 源IP: 206.1.1.2 -> 目的IP: 206.1.3.2 TTL=128 协议=1 (下一跳: 206.1.2.2) [转发] TTL -> 127 [转发] 将查询网关 (206.1.2.2) 的 MAC 地址。 [ARP缓存命中] 目标 MAC: 00:0c:29:3b:9b:36 [转发] 数据包已发送 [13][IPv4] 源IP: 206.1.3.2 -> 目的IP: 206.1.1.2 TTL=127 协议=1 (下一跳: 206.1.1.1) [转发] TTL -> 126 [转发] 将直接查询目标主机 (206.1.1.2) 的 MAC 地址。 [ARP缓存命中] 目标 MAC: 00:0c:29:3b:19:dd [转发] 数据包已发送 </pre>	<pre> [14][IPv4] 源IP: 206.1.1.2 -> 目的IP: 206.1.3.2 TTL=128 协议=1 (下一跳: 206.1.2.2) [转发] TTL -> 127 [转发] 将查询网关 (206.1.2.2) 的 MAC 地址。 [ARP缓存命中] 目标 MAC: 00:0c:29:3b:9b:36 [转发] 数据包已发送 [15][IPv4] 源IP: 206.1.3.2 -> 目的IP: 206.1.1.2 TTL=127 协议=1 (下一跳: 206.1.1.1) [转发] TTL -> 126 [转发] 将直接查询目标主机 (206.1.1.2) 的 MAC 地址。 [ARP缓存命中] 目标 MAC: 00:0c:29:3b:19:dd [转发] 数据包已发送 [16][IPv4] 源IP: 206.1.1.2 -> 目的IP: 206.1.3.2 TTL=128 协议=1 (下一跳: 206.1.2.2) [转发] TTL -> 127 [转发] 将查询网关 (206.1.2.2) 的 MAC 地址。 [ARP缓存命中] 目标 MAC: 00:0c:29:3b:9b:36 [转发] 数据包已发送 </pre>
--	---

图 5.19: 路由器程序抓包及转发日志

可以看到我们的路由器程序的工作日志，同样正确显示了数据报获取和转发过程，展示了数据报的源 IP、目的 IP、下一跳信息。同时也正确处理了 ARP 缓存表的命中过程。

综上，经过完整的测试，我们成功实现了该路由器程序，验证了该程序运行的正确性。